

AI-Assisted CTI Detection Engineering Platform

Technical Documentation & Deployment Guide

Internship Project

Forvis Mazars Group

Student: ILYES HAMDİ

Academic year: 2026 / 2027

Version: 1.0

Date: 2026-09-12

Technology stack: Python 3.11, FastAPI, SQLAlchemy, Alembic, PostgreSQL, Redis, Celery, LangGraph, MISP, DeepSeek, Next.js 14, TypeScript, Docker Compose, nginx

Table of Contents

1. Project Overview
2. System Architecture
3. CTI to Detection Engineering Pipeline
4. AI and LangGraph
5. Detection Engineering
6. Repository / Source Code Guide
7. Installation and Configuration
8. Running the Platform
9. API Documentation
10. Testing
11. DevSecOps / CI-CD
12. Troubleshooting
13. Security Considerations
14. Current Project Status
15. Documentation Index

1. Project Overview

This project implements an AI-assisted CTI detection engineering platform for a SOC-oriented local workflow. Its purpose is to convert MISP CTI events into validated, reviewable detection logic that can be represented as Sigma rules and then compiled into Splunk SPL. The platform is designed to be pragmatic, transparent, and auditable.

The system addresses a common operational problem: raw CTI often arrives as large, unstructured, noisy event data. The platform normalizes that data, extracts adversary behaviors, verifies ATT&CK; mappings, checks telemetry and coverage, decides whether a new detection is needed, generates a candidate rule, validates it, and only then presents it to a human analyst for approval.

The CTI source is MISP. The platform does not directly invent detections from generic threat data; it works from event-derived evidence, applies deterministic checks, and records the full workflow history in PostgreSQL.

The main roles in the system are:

- MISP: source of CTI data
- AI provider: behavior extraction and detection proposal support
- MITRE ATT&CK;: technique and tactic validation
- Sigma: structured detection language
- pySigma: translation to platform-target rule semantics
- human analyst: final approval or rejection

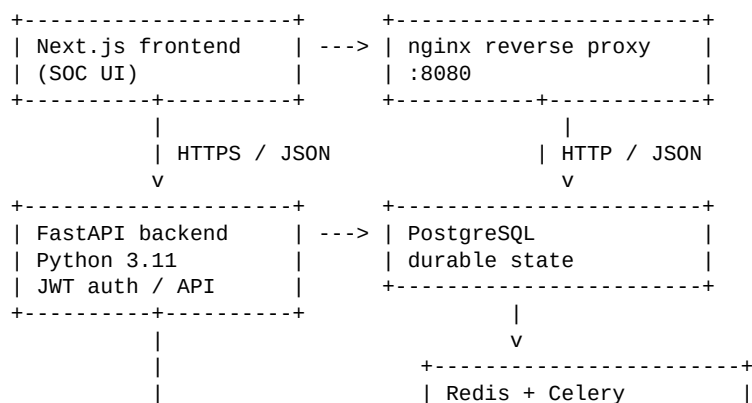
The complete workflow is:

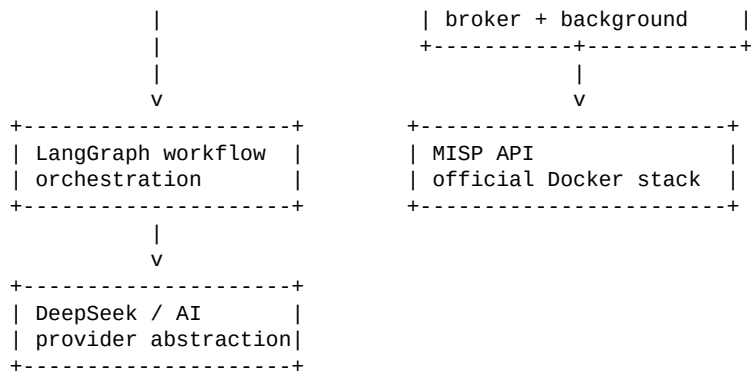
```
MISP CTI
-> Ingestion & normalization
-> Behavior extraction
-> MITRE ATT&CK mapping
-> Coverage / telemetry verification
-> Policy decision
-> AI-assisted detection generation
-> Sigma validation
-> pySigma compilation
-> Splunk SPL
-> Human review
-> Detection catalog
```

The repository implements this workflow with a clear boundary: AI proposes, deterministic services validate, and analysts decide.

2. System Architecture

The platform is composed of a real application stack and a local runtime environment.





Real runtime components

Next.js frontend

- UI for dashboards, threat queue, graph runs, AI workflow, detections, approvals, telemetry, and settings.
- Communicates with the backend API through `/api/v1`.
- Uses bearer-token authentication from a JWT returned by the backend.

FastAPI backend

- Exposes the application API, auth, dashboards, MISP endpoints, graph views, AI workflow views, proposals, detections, and settings.
- Validates access control using JWT and role checks.
- Is the central application logic layer.

PostgreSQL

- Durable source of truth for CTI events, behaviors, graph runs, proposals, validation results, review actions, and detection catalog data.
- Managed through SQLAlchemy models and Alembic migrations.

Redis and Celery

- Redis is used as the Celery broker and backend.
- Celery workers execute background tasks such as MISP polling and graph execution.
- Celery Beat schedules polling when configured.

LangGraph

- Implements the main graph-driven detection workflow.
- Persists node execution history and graph state.
- Determines routing based on validation and policy decisions.

MISP

- Official MISP Docker stack is used as the CTI source.
- Events are polled via the MISP API and normalized into the platform data model.
- The MISP profile is optional and controlled through Docker Compose.

AI provider / DeepSeek

- AI is abstracted behind a provider interface.
- The project supports deterministic fixture mode and a live DeepSeek integration when credentials are configured.

Nginx

- Reverse proxy for the frontend and API
- Exposes the app on `http://localhost:8080`

3. CTI to Detection Engineering Pipeline

The repository implements a real detection-engineering pipeline, not an abstract concept.

1. MISP ingestion

The platform pulls CTI from the MISP API. The MISP profile is configured in Docker Compose and can be activated with:

```
docker compose --profile misp up -d --build
```

The backend reads MISP data and stores event metadata, attributes, and normalized evidence.

2. CTI normalization

Raw MISP data is normalized into internal event records and workflow state. This step creates the structured model that downstream services can analyze consistently.

3. Behavior extraction

The graph extracts one or more suspicious, independently detectable behaviors from the normalized CTI. AI is used to produce candidate behaviors and structured evidence references.

4. ATT&CK; mapping

Candidate behaviors are mapped to ATT&CK; techniques and tactics. The platform then verifies that the mapping is valid, not revoked, and consistent with the available evidence.

5. Telemetry/visibility verification

The system checks whether the required telemetry sources exist and whether their data is available. This prevents the workflow from generating a rule that cannot be observed in the SOC environment.

6. Policy decision

The backend applies deterministic policy logic to decide whether a behavior is:

- already covered
- blocked by telemetry gaps
- lacking sufficient evidence
- eligible for candidate generation

7. AI-assisted detection generation

When policy allows it, the AI provider generates a Sigma candidate based on the CTI evidence and verified detection context.

8. Sigma validation

The Sigma candidate is checked using deterministic validation rules for structure, required fields, ATT&CK; tagging, and rule validity.

9. pySigma compilation

The project uses pySigma with the Splunk backend to compile the Sigma rule to Splunk SPL. That produces the deployable query representation used in detection engineering.

10. Repair loop

If validation fails and the issue is repairable, the graph calls a bounded repair path. The repair loop is capped by configuration and recorded in the database.

11. Analyst review

The final business decision is always human-controlled. An analyst can approve, request changes, or reject the proposal.

12. Publication/catalog

Only approved proposals become detection catalog entries and deployment artifacts. This is the publication step.

AI-assisted vs deterministic vs human-controlled

AI-assisted:

- behavior extraction
- ATT&CK proposals
- Sigma generation
- Sigma repair

Deterministic:

- ATT&CK verification
- coverage analysis
- telemetry checks
- duplicate detection
- Sigma validation
- pySigma compilation
- quality scoring
- policy decisions

Human-controlled:

- analyst approval
- request changes
- rejection
- final deployment publication

This distinction is important. The project does not claim that AI independently makes final security decisions.

4. AI and LangGraph

The intelligence layer is centered on LangGraph, which coordinates the workflow as a graph of node-based stages.

LangGraph workflow

```
consume_cti
-> extract_behaviors
-> verify_attack_mapping
-> coverage_analysis
-> visibility_analysis
-> policy_decision
-> generate_candidate
-> validate_candidate
-> repair_candidate
-> queue_review
-> approved
-> deployment
```

Graph nodes

The workflow contains explicit nodes such as:

- `consume\_cti`
- `extract\_behaviors`
- `verify\_attack\_mapping`
- `coverage\_analysis`
- `visibility\_analysis`
- `policy\_decision`
- `generate\_candidate`
- `validate\_candidate`
- `repair\_candidate`
- `queue\_review`
- `approved`
- `deployment`

The graph persists node execution inputs, outputs, retry counts, start/end times, durations, and failures. This creates an auditable record for each run.

AI provider abstraction

The backend uses an AI provider abstraction in `backend/app/services/ai.py`.

Supported modes include:

- deterministic fixture mode for demos and CI
- live DeepSeek integration when credentials are available

The interface standardizes behavior extraction and Sigma candidate generation so that the rest of the workflow is vendor-independent.

Structured AI outputs

The AI output is not freeform text only. The project uses structured objects, Pydantic schemas, and persisted JSON payloads for:

- behavior extraction outputs
- structured justification
- confidence information
- uncertainty metadata
- Sigma generation
- repair summaries

Validation and repair loop

After candidate generation, the platform validates:

- Sigma schema validity
- required fields and compatibility
- ATT&CK; integrity
- telemetry check
- duplicate detection
- quality score

If the candidate is invalid but repairable, the graph enters the repair loop. The repair loop is bounded to avoid endless AI iterations.

Confidence and reliability mechanisms

The system stores:

- AI confidence values
- watcher results
- trust/session memory
- revision history
- quality scores

This supports a reviewable decision trail even when AI is involved.

Demo and fixture mode

The project includes fixture-mode operation for local execution and repeatable testing. This is enabled by environment variables like `CTI\_AI\_FIXTURE\_MODE=true` in the sample environment files. This is a good demo and test mechanism, but it is not a production replacement for live AI capability.

Important limitation: live DeepSeek integration requires valid API credentials and an accessible network path.

5. Detection Engineering

Detection engineering in this project is the translation of CTI evidence into a reviewable Sigma rule that can be compiled to Splunk SPL.

MITRE ATT&CK;

The platform verifies ATT&CK; technique and tactic mapping using a local ATT&CK; reference model. It checks for valid technique references and rejects broken mappings before they proceed.

Telemetry

Telemetry checks answer the question: Is the necessary source data available in the platform environment? If the telemetry is missing or partial, the policy engine may stop the flow instead of generating a detection that cannot be supported.

Coverage

Coverage analysis compares new behavior against existing catalog entries and prior detections. This helps prevent duplication and identifies whether an event is already covered.

Sigma

Sigma is the structured detection language used to represent the candidate rule. It gives the project a portable, tool-agnostic detection representation.

pySigma

pySigma compiles the Sigma candidate into the backend target language. In this project, the configured target is Splunk. This is how the system produces Splunk SPL and validates the detection in a platform-specific form.

Splunk SPL

The compiled output is an executable query representation used by the downstream SOC environment. The project stores the compiled output and related validation artifacts in the database.

Duplicate detection

The project compares candidate logic and behavior fingerprints to detect near-duplicates or repeated detections. This is a key guardrail in a CTI engineering workflow.

Validation and repair

This project does not trust AI output without validation. Validation is deterministic and includes syntax, semantics, ATT&CK; consistency, coverage, telemetry, quality scoring, and duplicate checks. If the candidate fails but is repairable, it is revised within a bounded loop.

Why deterministic validation matters

Even when AI creates a candidate, the system still requires structured verification before anything is considered a valid detection. Security automation requires trustable evidence, not just plausible-looking text.

6. Repository / Source Code Guide

Repository tree

```
.  
■■■ .env.example
```

```
■■■ .github/
■   ■■■ workflows/
■■■ backend/
■   ■■■ alembic/
■   ■■■ app/
■   ■■■ Dockerfile
■   ■■■ pyproject.toml
■   ■■■ alembic.ini
■■■ docker/
■   ■■■ nginx.conf
■■■ docs/
■■■ frontend/
■   ■■■ app/
■   ■■■ components/
■   ■■■ lib/
■   ■■■ public/
■   ■■■ Dockerfile
■   ■■■ package.json
■   ■■■ next.config.mjs
■■■ scripts/
■   ■■■ backend/
■   ■■■ ci/
■■■ tests/
■   ■■■ e2e/
■   ■■■ integration/
■■■ docker-compose.yml
■■■ Makefile
■■■ README.md
■■■ sonar-project.properties
■■■ artifacts/
■■■ visual_audit_mockup1_20260806-150059/
```

Important directories

backend/

- Contains the FastAPI app, SQLAlchemy models, API routes, graph runner, service logic, worker tasks, and Alembic migration files.
- `backend/app/main.py` is the app factory and runtime entry point.
- `backend/app/graph/runner.py` contains the LangGraph orchestration logic.
- `backend/app/services/` holds the deterministic services for ATT&CK, coverage, telemetry, policy, Sigma validation, deployment, and MISP integration.

frontend/

- Contains the Next.js UI for dashboards, threat queue, review queue, AI workflow, detections, and system health.
- `frontend/lib/api.ts` is the HTTP client used for authenticated API calls.

tests/

- Contains integration and E2E checks to validate runtime behavior.
- The project includes integration tests for MISP-backed workflow execution and UI validation.

scripts/

- Contains script-based tests and CI/helper scripts.
- This is the actual backend validation path used by the repository.

.github/workflows/

- Contains CI/CD workflow definitions for quality, security, Docker validation, integration/E2E, and deployment hooks.

docs/

- Contains the technical documentation set.
- This folder is the source of the handoff package.

docker/

- Contains nginx configuration used for local reverse proxying.

7. Installation and Configuration

Prerequisites

The repository is configured around the following local toolchain:

- Git
- Docker and Docker Compose
- Python 3.11
- Node.js 20

The repository files show these versions and configurations:

- backend Docker image: `python:3.11-slim`
- frontend Docker image: `node:20-alpine`
- PostgreSQL: `postgres:16-alpine`
- Redis: `redis:7-alpine`

Clone the repository

```
git clone <repository-url>
cd <repository-name>
cp .env.example .env
```

Environment configuration

The sample environment file is `.env.example` and is the source of configuration. Use placeholders like:

```
CTI_ENVIRONMENT=local
CTI_DATABASE_URL=postgresql+psycopg://cti:cti@postgres:5432/cti
CTI_REDIS_URL=redis://redis:6379/0
CTI_JWT_SECRET=<your-token>
CTI_SEEDED_ADMIN_EMAIL=admin@example.com
CTI_SEEDED_ADMIN_PASSWORD=<your-password>
CTI_AI_FIXTURE_MODE=true
CTI_DEEPSEEK_API_KEY=<your-key>
CTI_MISP_URL=https://misp
CTI_MISP_API_KEY=<your-key>
CTI_MISP_VERIFY_TLS=false
CTI_MISP_POLL_INTERVAL_SECONDS=300
CTI_SIGMA_TARGET=splunk
CTI_MAX_REPAIR_ATTEMPTS=3
```

Required variables

- `CTI\_ENVIRONMENT`
- `CTI\_DATABASE\_URL`
- `CTI\_REDIS\_URL`
- `CTI\_JWT\_SECRET`
- `CTI\_SEEDED\_ADMIN\_EMAIL`
- `CTI\_SEEDED\_ADMIN\_PASSWORD`
- `CTI\_AI\_FIXTURE\_MODE`
- `CTI\_SIGMA\_TARGET`
- `CTI\_MAX\_REPAIR\_ATTEMPTS`

Optional variables

- `CTI\_DEEPSEEK\_API\_KEY`
- `CTI\_DEEPSEEK\_MODEL`
- `CTI\_MISP\_URL`
- `CTI\_MISP\_API\_KEY`
- `CTI\_MISP\_VERIFY\_TLS`
- `CTI\_MISP\_POLL\_INTERVAL\_SECONDS`

Development/demo variables

- `CTI\_AI\_FIXTURE\_MODE=true`
- local default admin credentials for demonstration use

External service credentials

- DeepSeek API key for live AI mode
- MISP API key for MISP polling
- deployment webhooks in CI/CD configuration

Never commit real secrets. Use placeholders or a developer-local secret store.

Docker setup

The repository is designed to run with Docker Compose. The standard command is:

```
docker compose up --build
```

The MISP-capable stack is enabled by:

```
docker compose --profile misp up -d --build
```

Database migration

The project uses Alembic. The verified working invocation is:

```
docker compose exec backend sh -lc "PYTHONPATH=/app alembic upgrade head"
```

This is the correct pattern for the current repository because the app is installed as a package in the backend container and the code expects `PYTHONPATH=/app`.

Starting services

To bring the full platform up:

```
docker compose up --build
```

To run the MISP-enabled stack:

```
docker compose --profile misp up -d --build
```

Health checks

The backend exposes:

```
GET /api/v1/health
```

The frontend is served by nginx on:

```
http://localhost:8080
```

The MISP UI is served on:

```
https://localhost:8443
```

Useful checks:

```
docker compose ps
curl http://localhost:8080/api/v1/health
```

And for PostgreSQL:

```
docker compose exec postgres pg_isready -U cti -d cti
```

And Redis:

```
docker compose exec redis redis-cli ping
```

8. Running the Platform

First-run walkthrough

1. Create a local environment file

```
cp .env.example .env
```

2. Add the required variables and secrets.

3. Start the base platform:

```
docker compose up --build
```

4. If MISP is required, start the MISP profile:

```
docker compose --profile misp up -d --build
```

5. Apply migrations:

```
docker compose exec backend sh -lc "PYTHONPATH=/app alembic upgrade head"
```

6. Verify backend health:

```
curl http://localhost:8080/api/v1/health
```

7. Open the frontend:

```
http://localhost:8080
```

8. Authenticate with the seeded admin credentials from `.env`.`

9. Access MISP and create or import a CTI event.

10. Trigger or inspect the workflow through the UI or API.

11. Review graph runs and AI workflow outputs.

12. Inspect ATT&CK; mappings, telemetry, and coverage.

13. Review the generated Sigma candidate and validation results.

14. Approve, request changes, or reject the proposal.

15. Confirm the approved detection reaches the detection catalog.

The actual workflow logic is implemented in the backend and graph runner; the UI is the analyst interaction layer.

9. API Documentation

Base path:

```
/api/v1
```

Authentication

- `POST /auth/login``
- `POST /auth/refresh``
- `GET /auth/me``

Dashboard

- `GET /dashboard/summary``
- `GET /dashboard/recent-decisions``

CTI and MISP

- `GET /cti-events``
- `GET /cti-events/{id}``

- `POST /cti-events/{id}/run-workflow`
- `POST /cti-events/{id}/reprocess`
- `GET /misp/events`
- `GET /misp/events/{misp\_event\_id}`
- `POST /misp/events/{misp\_event\_id}/ingest`
- `POST /misp/events/ingest-all-new`

Graph and AI

- `GET /graph-runs`
- `GET /graph-runs/{id}`
- `GET /graph-runs/{id}/timeline`
- `GET /ai/workflow`
- `GET /ai/dashboard`
- `GET /ai/sessions`
- `GET /ai/sessions/{id}`

Proposals and review

- `GET /proposals`
- `GET /proposals/{id}`
- `GET /proposals/{id}/revisions`
- `POST /proposals/{id}/approve`
- `POST /proposals/{id}/request-changes`
- `POST /proposals/{id}/reject`

Detections and telemetry

- `GET /detections`
- `GET /detections/{id}`
- `GET /attack/coverage`
- `GET /telemetry-sources`
- `GET /system-health`

Swagger OpenAPI

FastAPI exposes OpenAPI metadata automatically. The app can be inspected through the generated docs if the app is running.

10. Testing

The repository includes several test layers.

Backend tests

```
cd backend && pytest ../scripts/backend -q
```

The project has script-based backend tests covering AI contract validation, deterministic services, and MISP connectivity logic.

Integration tests

```
RUN_INTEGRATION_TESTS=1 python -m pytest tests/integration -q
```

These are runtime checks that verify the live stack and API behavior with the product running.

Frontend checks

```
cd frontend && npm run lint
cd frontend && npx tsc --noEmit
cd frontend && npm run build
```

Docker validation

```
docker compose config
docker compose build
docker compose up -d
```

Playwright E2E

```
npx playwright test tests/e2e --config=tests/e2e/playwright.config.ts
```

Security and static analysis

The GitHub workflows include:

- Ruff
- mypy
- Bandit
- pip-audit
- Semgrep
- Checkov
- Trivy
- ZAP baseline scan
- Snyk when configured
- SonarQube when configured

11. DevSecOps / CI-CD

The repository contains actual GitHub Actions workflows under ``.github/workflows/``.

Implemented and configured paths

- quality checks
- security scanning
- container validation
- integration and E2E automation
- deployment hooks

Workflows in the repo

- ``pr-quality.yml``
- ``security.yml``
- ``docker-validation.yml``
- ``integration-e2e.yml``
- ``deploy.yml``

Passing / implemented

These are defined in the repository and are intended to be enforced through GitHub Actions:

- backend unit checks
- Ruff lint and formatting
- mypy
- frontend lint and TypeScript compile
- frontend production build
- Docker Compose validation
- security scanning
- integration and E2E automation

Requires external configuration

- deployment webhooks
- Snyk token
- SonarQube secrets
- production environment configuration

Not currently enabled as production deployment

The project includes deployment hooks and workflow definitions, but it does not by itself constitute a production deployment stack for a real enterprise environment.

12. Troubleshooting

Docker containers failing

Check logs:

```
docker compose logs --no-color --tail=200 backend
```

Look for:

- missing ``.env``
- invalid PostgreSQL or Redis DSN
- port conflicts
- failed migration startup

Database connection issues

Verify PostgreSQL readiness:

```
docker compose exec postgres pg_isready -U cti -d cti
```

And inspect the DSN in ``.env``.

Alembic migrations

Use:

```
docker compose exec backend sh -lc "PYTHONPATH=/app alembic upgrade head"
```

If the command fails, verify the backend container working directory and the environment variable `PYTHONPATH=/app``.

Redis issues

```
docker compose exec redis redis-cli ping
```

Expected response:

```
PONG
```

MISP connectivity

Verify:

- `CTI_MISP_URL=https://misp``
- `CTI_MISP_API_KEY=``
- `CTI_MISP_VERIFY_TLS=false``
- `docker compose --profile misp ps``

Missing environment variables

The app reads environment variables using the `CTI_`` prefix. Missing variables can limit or break integrations.

DeepSeek / API issues

If `CTI_AI_FIXTURE_MODE=false`, a valid `CTI_DEEPSEEK_API_KEY` is needed. Otherwise the live AI path cannot operate reliably.

Frontend/backend connectivity

The frontend is proxied by nginx on port 8080. Verify backend health with:

```
curl http://localhost:8080/api/v1/health
```

Port conflicts

Common ports in this repo:

- `8080` — nginx frontend proxy
- `8443` — MISP HTTPS interface
- `5432` — PostgreSQL service
- `6379` — Redis service

13. Security Considerations

Secret handling

The project expects secrets to be managed locally or in a secure environment, especially:

- `CTI_JWT_SECRET`
- `CTI_SEEDED_ADMIN_PASSWORD`
- `CTI_MISP_API_KEY`
- `CTI_DEEPSEEK_API_KEY`

Authentication and authorization

The backend uses JWTs and role-based access checks. The seeded admin user is created from environment variables. This is a useful local mechanism but not a hardened enterprise identity model.

Container isolation

The project uses separate Docker networks and a dedicated MISP profile. This is appropriate for local demo separation, but it is not an enterprise network topology.

Dependency and code scanning

The repository includes SAST and dependency scanning through GitHub workflows, including:

- Ruff
- Bandit
- Semgrep
- pip-audit

- Trivy
- Snyk (when configured)
- SonarQube (when configured)

DAST and container scanning

OWASP ZAP baseline scan is part of the CI flow, and container security scanning is also configured in the project workflows.

Known limitations and production hardening requirements

- wildcard CORS is convenient for local development but not production-safe
- JWTs and tokens should be managed with a secret manager in real deployments
- frontend token storage via browser localStorage is a demo-friendly pattern, not a hardened production model
- no enterprise SSO or MFA layer is implemented
- MISP is local profile-based and not equivalent to a hardened production MISP installation
- AI provider connectivity remains external and must be credentialed
- the project is a functional engineering prototype, not a full production SOC platform

14. Current Project Status

Implemented

- MISP ingestion and normalization
- LangGraph orchestration
- AI-assisted behavior extraction and Sigma generation
- deterministic validation and repair loop
- analyst review workflow
- PostgreSQL-backed state tracking
- Docker Compose runtime
- local developer and CI security workflow

Partially implemented

- some production-style deploy infrastructure exists, but deployment secrets/configuration are external
- MISP runtime is available as an optional local stack, not as a hardened enterprise deployment
- E2E coverage is useful for smoke validation rather than full enterprise coverage

Requires external configuration

- DeepSeek API credentials for live AI mode
- MISP API key for live MISP polling
- deployment webhooks

- Snyk token and SonarQube secrets

Known limitations

- local defaults are intended for development and demo
- production hardening remains incomplete
- no enterprise identity provider or MFA layer
- local AI fixture mode is not a production-ready replacement

Production hardening requirements

- explicit CORS rules
- secret management system
- secure session handling
- production-grade monitoring and alerting
- enterprise identity integration
- stronger governance around approval and release policy

15. Documentation Index

- README.md
- docs/ARCHITECTURE.md
- docs/SOURCE_CODE_GUIDE.md
- docs/RUNNING_THE_PROJECT.md
- docs/API.md
- docs/DATABASE.md
- docs/AI_PIPELINE.md
- docs/DETECTION_ENGINEERING.md
- docs/DEVSECOPS_PIPELINE.md
- docs/MISP_INTEGRATION.md
- docs/SECURITY_AND_LIMITATIONS.md

The markdown documentation in the repository is the source of the handoff package, and the PDF export is intended to provide a professional single-document version for the supervisor.